



14. Python: Strings

"Everything is an object, and strings are immutable."

Previous Section:

 [13. Python: Dictionaries](#)

Strings are another hard-to-overlook data structure in Python. Almost everything we do in programming involves text in some way — names, messages, passwords, file paths, user input, and even data from websites. Because of that, understanding strings is extremely important.

In simple terms, a string is a sequence of characters stored together as one piece of data. These characters can include letters, numbers, symbols, spaces, or even special characters.

For example:

```
"Hello Iris"  
"Python123"  
"@Iris Academy"
```

Unlike numbers, strings are meant to represent text and information rather than perform mathematical calculations. Because of this, Python treats them differently from integers and floats.

1. What is a String?

Even though we already know what text is in real life, in programming we need a more precise definition.

A string is a collection of characters enclosed inside quotation marks. In Python, we usually use either single quotes `' '` or double quotes `" "` to create

strings.

```
name = "Iris"
message = 'Hello World'
```

The quotation marks are very important because they tell Python: ***"This is text, not a variable name."***

Without quotes, Python will think the word is a variable instead of a string.

```
name = Iris
```

This produces an error because Python tries to find a variable called `Iris`.

Strings vs Numbers

One thing beginners often misunderstand is this: ***A string can look like a number, but it is still not a real number.***

```
x = "100"    # String
y = 100      # Integer
```

Even though both contain `100`, they behave very differently.

The biggest difference appears when using the `+` operator. With numbers, `+` performs addition:

```
a = 10
b = 20
print(a + b)
```

30

But with strings, `+` performs concatenation (joining text together):

```
a = "10"  
b = "20"  
print(a + b)
```

1020

Python joins the two strings instead of calculating them mathematically.
Another example:

```
first_name = "Iris"  
last_name = "Academy"  
  
print(first_name + " " + last_name)
```

Iris Academy

So even though strings may contain numbers, they are still treated as text data rather than values for calculations.

Single Quotes vs Double Quotes

As mentioned earlier: **In Python, both single quotes `' '` and double quotes `" "` work the same way for creating strings.**

Example:

```
name1 = 'Iris'  
name2 = "Iris"
```

Both are completely valid. But here is a small catch: ***If we randomly mix single and double quotes everywhere, our code can become messy very quickly.***

Because of that, my recommendation is simple: Pick one style consistently. Personally, I recommend using single quotes `' '` for most normal strings. Why? Because it helps us avoid problems when displaying quotation marks inside text.

Example:

```
message = "I am 'excellent'"  
print(message)
```

I am 'excellent'

This is much cleaner than struggling with escape characters like:

```
message = 'I am \'excellent\''
```

Technically this still works, but it makes the code harder to read.

A Better Modern Choice: f-Strings

In modern Python, a much better solution is using f-strings. Example:

```
name = 'Iris'  
message = f'My name is "{name}"'  
print(message)
```

My name is "Iris"

f-strings make strings cleaner, easier to read, and much more powerful when combining variables with text.

So in practice: Use a consistent quote style. Single quotes are usually cleaner for normal strings. Use f-strings whenever variables are involved. This keeps our code readable and avoids unnecessary quotation problems later.

2. Operations on Strings

Strings are one of the most flexible data types in Python. Unlike numbers, strings are mainly used to store and manipulate text. Python provides many built-in operations and methods that allow us to work with strings efficiently.

String Variables

A string in Python can contain almost anything: Letters; Numbers; Symbols; Spaces; Special characters

```
# String in Python - String can be anything
string1 = "Hello World"
string2 = "Welcome to my world"
string3 = "12"

print(string1)
print(string2)
print(string3)
```

```
Hello World
Welcome to my world
12
```

- We can also convert numbers into strings using `str()`.

```
number = 100

print(str(number))
print(type(str(number)))
```

```
100
<class 'str'>
```

- However, methods like `int()` or `float()` only work if the string itself contains valid numeric values.

```
print(int("25"))
print(float("3.14"))

# Invalid conversion
print(int("Hello"))
```

25

3.14

ValueError: invalid literal for int() with base 10: 'Hello'

The last conversion produces an error because `"Hello"` is not a number.

Addition and Multiplication for Strings

Strings support special operations using `+` and `*`.

- String Concatenation (`+`): The `+` operator joins strings together.

```
string1 = "Hello, and Welcome to"
string2 = " My World"

print(string1 + string2)
```

Hello, and Welcome to My World

- String Repetition (`*`): The `*` operator repeats a string multiple times.

```
string3 = "Python "
print(string3 * 5)
```

Python Python Python Python Python

This is useful when printing patterns, separators, or repeated messages.

Indexing and Slicing in a String

Each character inside a string has its own position called an index. Python indexing starts from 0.

```
string = "Welcome to my world, welcome to Python"

print("Length:", len(string)) # Includes spaces
print("Length (no spaces):", len(string.replace(" ", "")))

print(string[0:4])           # Characters from index 0 to 3
print(string[2:24])          # Characters from index 2 to 23
print(string[:-1])           # Everything except the last character
```

Length: 41

Length (no spaces): 35

Welc

lcome to my world, we

Welcome to my world, welcome to Pytho

Comparing Strings

Strings can be compared using comparison operators. Python compares strings based on their Unicode (ASCII) values.

```

string1 = "Hello World"
string2 = "Welcome to my world"
print(string1 == string2)
print(string1 != string2)
print(string1 > string2)
print(string1 < string2)

string3 = "Hello world"
print(string1 == string3) # Case-sensitive
print(string1 > string3)
print(string1 < string3)

```

```

False
True
False
True
False
True
False

```

Notice that uppercase and lowercase letters are treated differently.

Substring

A substring is simply a smaller portion extracted from a larger string. We can check whether a substring exists using the `in` keyword.

```

string1 = "Hello World"
string2 = "Welcome to my World"

print("World" in string1)
print("World" in string2)
print("Hello World" in string2)

```

```

True
True
False

```


This is commonly used in searching and validation.

Join a List into a String

The `join()` method combines elements from a list into a single string using a separator.

```
month_dict = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4,
              'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8,
              'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12}

month_list = list(month_dict.keys())
print(', '.join(str(i) for i in month_list))
```

Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec

Splitting Strings

The `split()` method breaks a string into smaller parts based on a delimiter.

```
string = "Welcome to my world, welcome to Python"
# Count words
print(len(string.split()))
```

7

Using `split()` for Multiple Inputs

The `split()` method is extremely useful for handling user input.

```
def display_student_info(name, age, city, major):
    print("-" * 20)
    print("Name:", name)
    print("Age:", age)
    print("City:", city)
    print("Major:", major)
    print("-" * 20)

name, age, city = input("Enter your name, age, and city: ").split(",")
major = input("Enter your major: ")
display_student_info(name, age, city, major)
```

Enter your name, age, and city: Alice,18,Tokyo

Enter your major: Computer Science

Name: Alice

Age: 18

City: Tokyo

Major: Computer Science

Replacing Strings

The `replace()` method replaces one substring with another.

```
string1 = "Hello World"
print(string1.replace("World", "Python"))

string2 = "welcome to my world, Welcome to Python"
print(string2.replace("welcome to", "Hi"))
```

Hello Python

Hi my world, Welcome to Python

Unicode in Python

Every character in Python has a Unicode value.

- `ord()` → Converts character to Unicode number
- `chr()` → Converts Unicode number back to character

```
import string

# Unicode values from a to z
string_list = list(string.ascii_lowercase)

count = 0

for i in string_list:
    print(ord(i), end=", ")
    count += 1

    if count == 5:
        print()
        count = 0

print()

# Characters from Unicode 50 to 65
for j in range(50, 66):
    print(chr(j), end=", ")
```

```
97, 98, 99, 100, 101,
102, 103, 104, 105, 106,
107, 108, 109, 110, 111,
112, 113, 114, 115, 116,
117, 118, 119, 120, 121,
122,
2, 3, 4, 5, 6, 7, 8, 9, :, ;, <, =, >, ?, @, A,
```

Unicode is extremely important because computers internally store characters as numbers.

Other Methods on Strings

Python provides many useful built-in string methods.

- `upper()`, `lower()`, `capitalize()`: These methods modify letter casing.

```
string = "I am Alice, 29 years old"

print("Uppercase:", string.upper())
print("Lowercase:", string.lower())
print("Capitalized:", string.capitalize())
```

Uppercase: I AM ALICE, 29 YEARS OLD

Lowercase: i am alice, 29 years old

Capitalized: I am alice, 29 years old

- `find()`, `startswith()`, `endswith()`: These methods help us search inside strings.

```
print("Index of 'Alice':", string.find("Alice"))
print("Starts with 'I am':", string.startswith("I am"))
print("Ends with 'Python':", string.endswith("Python"))
```

Index of 'Alice': 5

Starts with 'I am': True

Ends with 'Python': False

- `count()`, `isalpha()`, `isdigit()`, `isalnum()`: These methods help validate string content.

```

print("Count of 'a':", string.count("a"))

sample1 = "Alice"
sample2 = "123"
sample3 = "Alice123"

print(f'{sample1}' only contains letters:", sample1.isalpha())
print(f'{sample2}' only contains digits:", sample2.isdigit())
print(f'{sample3}' contains letters & digits:", sample3.isalnum())

```

```

Count of 'a': 1
'Alice' only contains letters: True
'123' only contains digits: True
'Alice123' contains letters & digits: True

```

Summary

Strings are one of the most versatile and important data types in Python.

They allow us to:

- Store and manipulate text
- Combine and repeat information
- Access characters using indexing and slicing
- Compare text values
- Search for substrings
- Split and join data
- Replace content dynamically
- Work with Unicode characters
- Validate and format text easily

Because text appears almost everywhere in programming, mastering strings is an essential skill for every Python programmer.

References

<https://www.geeksforgeeks.org/python/python-string/>

<https://developers.google.com/edu/python/strings>

<https://www.codecademy.com/learn/dacp-python-fundamentals/modules/dscp-python-strings/cheatsheet>

<https://www.programiz.com/python-programming/string>

Next Section:

[`</>` 15. Python: Mathematical Functions](#)